LAB MANUAL
PART A
(PART A : TO BE REFERRED BY STUDENTS)

Experiment no: - 1

Experiment Name: - DDL statements.

Aim: - Performing practical by using DDL statements (creating tables; managing tables, constraints).

Resource required: Oracle SQL Developer

Theory: -

CREATING AND MANAGING TABLES:

- Table is a basic unit of storage that composed of rows and columns.

1. **CREATE TABLE Statement**:
   create The **CREATE TABLE** statement is used to create a new table in the database.

Syntax

```
CREATE TABLE table_name
(
   Column_name1  datatype ,
   Column_name2  datatype
   ...
);
```

EXAMPLE -

```
CREATE TABLE dept
(
   deptno NUMBER(2),
   dname  VARCHAR2(14),
   loc    VARCHAR2(13)
);
```

2. Verify the Table Structure

Use the **DESCRIBE** command to view the table structure.

DESCRIBE dept;

Tables in the Oracle Database:

User Tables:

- Collection of tables created and maintained by the user. Contain user information

Data Dictionary:

- Collection of tables created and maintained by the Oracle Server. Contain database information

2. **ALTER TABLE Statement:**

- use to add, modify, define values and drop a column.

Syntax:

1) ALTER TABLE table
ADD (column datatype [DEFAULT expr] [, column datatype] …);

2) ALTER TABLE table
MODIFY (column datatype [DEFAULT expr] [, column datatype] …);

3) ALTER TABLE table
DROP (column datatype [DEFAULT expr] [, column datatype] …);tes at the Data Link Layer.

Uses MAC addresses to forward frames.
Connects devices within the same LAN.
Reduces collisions compared to hubs.
2. Router (Layer 3 Device)
Operates at the Network Layer.
Uses IP addresses to route packets between networks.
Separates broadcast domains.
Enables communication between different LANs.
3. End Devices
PCs and laptops act as hosts.
Each must have:
IP address
Subnet mask
Default gateway (router IP)
Communication Flow Example
If PC0 wants to send data to PC1:
Data stays within the switch (same LAN).
If PC0 wants to send data to Laptop0:
Data goes:

e.g.: 1) **Add a column:**

```
ALTER TABLE
dept80
ADD    (job_id VARCHAR2 (9));
```

2) **Drop a column:**

```
ALTER TABLE dept80
MODIFY (last_name VARCHAR2 (30));
```

3) **Drop a column:**

```
ALTER TABLE dept80
DROP COLUMN job_id;
```

3. **Dropping a Table:** table is deleted
e.g.: DROP TABLE dept80;

4. **Truncating a Table:**
Syntax: TRUNCATE TABLE detail_dept;

- **CONSTRAINTS:**
  - Constraints enforce rules at the table level.
  - Constraints prevent the deletion of a table if there if there are dependencies.

Defining Constraints:

```
Syntax:  CREATE TABLE [schema.] table
                        (column datatype [DEFAULT
                         expr] [ column_constraint],
                            …
                        [table_constraint] [, …]
); e.g.: CREATE TABLE employees (
                    employee_id   NUMBER (6),
                   first_name     VARCHAR2 (20),
                            …
                    job_id         VARCHAR2 (10) NOT NULL
                   CONSTRAINT emp_emp_id-pk
                                PRIMARY KEY (EMPLOYEE_ID));
```

Types of Constraint

1. **NOT NULL:** defining at the column level e.g.: CREATE TABLE employees (

```
                        employee_id        NUMBER (6),
                        last_name          VARCHAR2 (25) NOT NULL,
                        hire_date          DATE
                                           CONSTRAINT emp_hire_date_nn
                                            NOT NULL,
```

2. **UNIQUE Constraint**: defined at either the table level or the column level. e.g.: CREATE TABLE employees (

```
                        employee_id        NUMBER (6),
```

```
                                last_name           VARCHAR2 (25) NOT NULL,
                                email               VARCHAR2 (25),
                                hire_date           DATE,
                …
                    CONSTRAINT emp_email_uk UNIQUE (email));
```

3. **PRIMARY Constraint:** defined at either the table level or the column level

```
e.g.: CREATE TABLE      departments (
                        department_id       NUMBER (4),
                        department_name     VARCHAR2 (30)
                        manager_id          NUMBER (6),
                CONSTRAINT dept_id_pk PRIMARY KEY (department_id));
```

4. **FOREIGN KEY Constraint:** defined at either the table level or the column level

A **Foreign Key (FK)** is a column in one table that refers to the **Primary Key (PK)** of another table. It creates a relationship between two tables and prevents invalid data from being entered.

e.g.:

```
CREATE TABLE Department (
dept_id NUMBER(2) PRIMARY KEY,
dept_name VARCHAR2(20)
);

CREATE TABLE Employee (
emp_id NUMBER(4) PRIMARY KEY,
emp_name VARCHAR2(30),
dept_id NUMBER(2),
CONSTRAINT fk_dept
FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
 );
```

5. **CHECK Constraint**: define a condition that each row must be satisfy e.g.:

```
                … salary        NUMBER (2)
            CONSTRAINT emp_salary_min
            CHECK (salary &amp;gt; 0)…
```

Adding a Constraint:

```
Syntax:   ALTER TABLE table
          ADD [CONSTRAINT constraint] type (column);
          e.g.:       ALTER TABLE        employees
      ADD CONSTRAINT emp-manager_fk
        FOREIGN KEY     (manager_id)
```

```
    PREFERENCES    employees
    (employee_id);
```

Dropping Constraint:

```
ALTER TABLE       employees
DROP CONSTRAINT      emp-manager-fk;
```

Conclusion:

In this practical, learned how to create and manage tables and adding constraints to existing tables

PART B
(PART B: TO BE COMPLETED BY STUDENTS)

Roll. No: B44	Name: ARYAN VIKAS SHINDE
Class: B(SE-COMPS)	Batch: B2
Date of Experiment:	Date of Submission:
Grade:	

B.1: Read the given question carefully and write a SQL statement and check the output.

(Put the query and its output after each question.)

Form a group of **2–3 students** and perform the following tasks:

- Identify and define a clear **problem statement**

GYM MANAGEMENT SYSTEM – Brief Description

A Gym Management System is a database-based application used to manage information about gym members, trainers, memberships, and workouts in a centralized database. It stores details such as member information, trainer details, membership plans, and workout programs. Using a DBMS, the system helps manage gym activities, maintain organized records, reduce data redundancy, and provide efficient access to gym-related information.

- Create the required **database tables**.

ENTITIES	ATTRIBUTES	CONSTRAINTS
MEMBER	Member_ID Member_Name Phone Age	Primary key (PK), NOT NULL NOT NULL UNIQUE, NOT NULL CHECK (Age >= 15)
TRAINER	Trainer_ID Trainer_Name Specialization Phone	Primary Key (PK), NOT NULL NOT NUL NOT NUL UNIQUE, NOT NULL
MEMBERSHIP	Membership_ID Member_ID Membership_Type Fee	Primary key (PK), NOT NULL Foreign key (FK), NOT NULL NOT NULL CHECK (Fee > 0)
WORKOUT	Workout_ID Workout_Name Duration Difficulty_Level	Primary Key (PK), NOT NULL NOT NULL CHECK(duration>0) NOT NULL

INPUT:-

[SQL Worksheet]*

```
1   CREATE TABLE MEMBER
2   (
3       Member_ID NUMBER(5) PRIMARY KEY,
4       Member_Name VARCHAR2(30) NOT NULL,
5       Phone VARCHAR2(15) UNIQUE,
6       Age NUMBER(3) CHECK (Age >= 15)
7   );
8
9   CREATE TABLE TRAINER
10  (
11      Trainer_ID NUMBER(5) PRIMARY KEY,
12      Trainer_Name VARCHAR2(30) NOT NULL,
13      Specialization VARCHAR2(30) NOT NULL,
14      Phone VARCHAR2(15) UNIQUE
15  );
16
17  CREATE TABLE MEMBERSHIP
18  (
19      Membership_ID NUMBER(5) PRIMARY KEY,
20      Member_ID NUMBER(5) NOT NULL,
21      Membership_Type VARCHAR2(20) NOT NULL,
22      Fee NUMBER(8,2) CHECK (Fee > 0),
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	MEMBER_ID	MEMBER_NAME	PHONE	AGE
1	101	Rahul	9876543210	22
2	102	Amit	9876543211	25
3	103	Sneha	9876543212	21

[SQL Worksheet]*

```
18      (
23          FOREIGN KEY (Member_ID) REFERENCES MEMBER(Member_ID)
24      );
25
26      CREATE TABLE WORKOUT
27      (
28          Workout_ID NUMBER(5) PRIMARY KEY,
29          Workout_Name VARCHAR2(30) NOT NULL,
30          Duration NUMBER(3) CHECK (Duration > 0),
31          Difficulty_Level VARCHAR2(15) NOT NULL
32      );
33
34      INSERT INTO MEMBER VALUES (101, 'Rahul', '9876543210', 22);
35      INSERT INTO MEMBER VALUES (102, 'Amit', '9876543211', 25);
36      INSERT INTO MEMBER VALUES (103, 'Sneha', '9876543212', 21);
37
38      INSERT INTO TRAINER VALUES (201, 'Vikram', 'Weight Training', '9876500011');
39      INSERT INTO TRAINER VALUES (202, 'Neha', 'Yoga', '9876500012');
40      INSERT INTO TRAINER VALUES (203, 'Akash', 'Cardio', '9876500013');
41
42      INSERT INTO MEMBERSHIP VALUES (301, 101, 'Monthly', 1500);
43      INSERT INTO MEMBERSHIP VALUES (302, 102, 'Quarterly', 4000);
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download | Execution time: 0.003 seconds

	MEMBER_ID	MEMBER_NAME	PHONE	AGE
1	101	Rahul	9876543210	22
2	102	Amit	9876543211	25
3	103	Sneha	9876543212	21

[SQL Worksheet]*

```
41
42    INSERT INTO MEMBERSHIP VALUES (301, 101, 'Monthly', 1500);
43    INSERT INTO MEMBERSHIP VALUES (302, 102, 'Quarterly', 4000);
44    INSERT INTO MEMBERSHIP VALUES (303, 103, 'Yearly', 12000);
45
46    INSERT INTO WORKOUT VALUES (401, 'Weight Training', 60, 'Hard');
47    INSERT INTO WORKOUT VALUES (402, 'Yoga', 45, 'Easy');
48    INSERT INTO WORKOUT VALUES (403, 'Cardio', 30, 'Medium');
49
50    SELECT * FROM MEMBER;
51    SELECT * FROM TRAINER;
52    SELECT * FROM MEMBERSHIP;
53    SELECT * FROM WORKOUT;
54
55    ALTER TABLE MEMBER ADD Email VARCHAR2(40);
56    ALTER TABLE MEMBER MODIFY Email VARCHAR2(50);
57    ALTER TABLE MEMBER DROP COLUMN Email;
58    TRUNCATE TABLE WORKOUT;
59    DROP TABLE WORKOUT;
```

Query result | Script output | DBMS output | Explain Plan | SQL history

Download — Execution time: 0.003 seconds

	MEMBER_ID	MEMBER_NAME	PHONE	AGE
1	101	Rahul	9876543210	22
2	102	Amit	9876543211	25
3	103	Sneha	9876543212	21

B.2: Write an observation according to the query execution

Observation :

The SQL queries were executed successfully, and all the required operations were completed without errors. The CREATE, ALTER, DROP, and TRUNCATE commands helped in understanding how to create, modify, delete, and manage database tables. This practical improved knowledge of SQL commands used in DBMS.

B.3: Write an answer of the following questions

1. What is the difference between the `CREATE` and `ALTER` statements?

   The CREATE statement is used to create a new database object such as a table, database, or view. It defines the structure of the object before storing data.

   The ALTER statement is used to make changes to an existing database object. It allows users to add, remove, or modify columns and constraints without creating a new table.

   In short: CREATE creates a new object, while ALTER changes the structure of an existing object.

2. Explain the difference between `DROP` and `TRUNCATE` statements.

   The DROP statement permanently deletes a database object, such as a table, along with all its data and structure. Once dropped, the table must be created again before it can be used.

   The TRUNCATE statement removes all records from a table but keeps the table structure unchanged. It clears the data quickly so that new records can be inserted later.

   In short: DROP deletes both the table and its structure, whereas TRUNCATE removes only the data and keeps the table intact.

3. What happens when you use the `DROP TABLE` command?

The DROP statement permanently deletes a database object, such as a table, along with all its data and structure. Once dropped, the table must be created again before it can be used. The TRUNCATE statement removes all records from a table but keeps the table structure unchanged. It clears the data quickly so that new records can be inserted later.
In short: DROP deletes both the table and its structure, whereas TRUNCATE removes only the data and keeps the table intact.